

JENKINS CI PIPELINE PROJECT

Part 2 – New Jenkins

Step-by-step guide with one command at a time

This guide continues after Part 1. It is based on your PART2 JENKINS document, with the commands arranged clearly so you can copy and run them one by one.

Important: Copy one complete command, paste it into the Ubuntu terminal, press Enter, wait for it to finish, and only then run the next command.

Project flow: GitHub → Jenkins → Maven Build → Docker → Docker Hub → Pipeline Result

Repositories used:

Shared Library:

<https://github.com/DEVOPS-WITH-WEB-DEV/jenkins-shared-library1.git>

Project Repository:

<https://github.com/DEVOPS-WITH-WEB-DEV/spring-cloud-kubernetes.git>

STEP 0 – CHECK JENKINS FIRST

Confirm that Jenkins is running:

```
sudo systemctl status jenkins --no-pager
```

Your earlier error showed Java 17 while your Jenkins installation required Java 21. If Jenkins still does not start, run:

```
sudo apt update
```

```
sudo apt install openjdk-21-jdk -y
```

```
java -version
```

```
sudo systemctl restart jenkins
```

```
sudo systemctl status jenkins --no-pager
```

Continue only when Jenkins shows **active (running)**.

STEP 1 – INSTALL DOCKER

```
sudo apt update  
sudo apt install docker.io -y  
docker --version  
sudo systemctl start docker  
sudo systemctl enable docker  
sudo systemctl status docker --no-pager
```

STEP 2 – ALLOW JENKINS TO USE DOCKER

Instead of using `chmod 777` on the Docker socket, add the Jenkins user to the Docker group:

```
sudo usermod -aG docker jenkins  
sudo systemctl restart jenkins
```

Test Jenkins Docker access:

```
sudo -u jenkins docker ps
```

An empty container list means Docker is working and there are currently no containers.

STEP 3 – ADD DOCKER HUB CREDENTIALS

Open Jenkins in your browser:

`http://YOUR_EC2_PUBLIC_IP:8080`

Go step by step:

1. Manage Jenkins
2. Credentials
3. System
4. Global credentials (unrestricted)
5. Add Credentials

Add your Docker Hub username and password/access token. Use this Credential ID:

`docker_cred`

Save the credentials.

STEP 4 – ADD MAVEN

Go to: Manage Jenkins → Tools → Maven installations → Add Maven

Recommended:

Name: Maven

Enable: Install automatically

Click Save

If the Jenkinsfile expects another Maven name, use the exact name used by that Jenkinsfile.

STEP 5 – ADD JENKINS SHARED LIBRARY

Go to: Manage Jenkins → Configure System → Global Pipeline Libraries

Library Name	jenkins-shared-library
Default Version	main
Repository	https://github.com/DEVOPS-WITH-WEB-DEV/jenkins-shared-library1.git

Save the configuration.

STEP 6 – CREATE THE JENKINS PIPELINE

1. Jenkins Dashboard → New Item
2. Pipeline name: Spring-Cloud-Kubernetes-CI
3. Select Pipeline
4. Click OK
5. Under Pipeline select: Pipeline script from SCM
6. Select SCM: Git

Repository URL:

`https://github.com/DEVOPS-WITH-WEB-DEV/spring-cloud-kubernetes.git`

Branch:

`*/main`

Script Path:

`kubernetes-configmap-reload/Jenkinsfile`

Click Save.

STEP 7 – BUILD THE PIPELINE

Open your pipeline and click:

Build Now

Then open the build number → Console Output.

The configured pipeline may perform source checkout, Maven build, Docker image build, Docker Hub login/push, and deployment-related steps. The exact stages depend on the Jenkinsfile and shared library.

STEP 8 – TROUBLESHOOTING

Docker permission problem:

```
sudo usermod -aG docker jenkins
```

```
sudo systemctl restart jenkins
```

```
sudo -u jenkins docker ps
```

Jenkins problem:

```
java -version
```

```
sudo journalctl -u jenkins.service -n 50 --no-pager
```

```
sudo systemctl status jenkins --no-pager
```

Docker service problem:

```
sudo systemctl status docker --no-pager
```

FINAL CHECKLIST

- Jenkins is active and running
- Java version is compatible with Jenkins
- Docker is installed and running
- Jenkins user can access Docker
- Docker Hub credentials are added
- Credential ID is correct
- Maven is configured
- Shared library is configured
- Project repository is configured
- Branch is */main
- Jenkinsfile path is correct
- Pipeline is saved
- Build is started